



HW04 Code Review

張皓庭

HW04 Review

- 大量矩陣乘法
 - single-head mode
 - two-head mode
- register 可以不用全部存下來
- 硬體共用 → 優化

● single-head mode (mode = 0)

- You will receive Q [8×8] and K [8×8], then you have to compute

$$S[8 \times 8] = Q[8 \times 8] \times K^T[8 \times 8]$$

Q								K								S = Q * K ^T							
-2	-7	5	-2	3	-7	-8	-1	-3	-4	-2	-6	3	-2	3	3	32	25	147	-103	-47	-23	-106	-50
6	-8	4	7	-1	4	-3	6	6	-6	3	2	3	-3	6	-2	-38	65	-57	66	-9	-69	-48	-115
3	-5	-8	0	4	2	-7	-2	-1	-8	-5	-6	6	-8	-3	-4	8	-8	114	-21	13	-9	22	10
7	-6	0	7	-4	3	-3	-6	4	0	-4	7	-6	0	5	3	-84	65	-16	68	-32	-86	-17	3
2	1	0	7	-4	-5	-1	1	1	6	-5	7	4	-4	0	4	-54	15	-37	79	65	-18	6	-68
4	-3	5	0	5	5	2	-8	4	1	-4	-6	3	-6	6	3	-23	85	11	-48	-71	-34	-89	32
3	2	-2	3	4	4	1	3	5	5	-8	-5	-7	1	0	3	-15	6	-50	31	58	7	11	-56
-6	7	-3	6	-1	-1	-1	-2	-4	-3	-5	-7	-5	3	5	-8	-50	-77	-58	25	85	-50	-1	-11

- Apply the activation function **act(·)** element-wise to S, then you get P,

$$\text{where } \text{act}(x) = \begin{cases} x, & x \geq 0 \\ x/4 \text{ (truncate toward zero)}, & \text{otherwise} \end{cases}$$

S = Q * K ^T								P = act(S)							
32	25	147	-103	-47	-23	-106	-50	32	25	147	-25	-11	-5	-26	-12
-38	65	-57	66	-9	-69	-48	-115	-9	65	-14	66	-2	-17	-12	-28
8	-8	114	-21	13	-9	22	10	8	-2	114	-5	13	-2	22	10
-84	65	-16	68	-32	-86	-17	3	-21	65	-4	68	-8	-21	-4	3
-54	15	-37	79	65	-18	6	-68	-13	15	-9	79	65	-4	6	-17
-23	85	11	-48	-71	-34	-89	32	-5	85	11	-12	-17	-8	-22	32
-15	6	-50	31	58	7	11	-56	-3	6	-12	31	58	7	11	-14
-50	-77	-58	25	85	-50	-1	-11	-12	-19	-14	25	85	-12	0	-2

- Using `get_value` to get V[8×8].

Then compute the output matrix: **out_data** [8×8] = **O**[8×8] = **P**[8×8] × **V**[8×8]

P = act(S)								V								out_data = P * V							
32	25	147	-25	-11	-5	-26	-12	5	-7	-6	1	-5	-7	-3	0	-93	-642	-554	-748	186	-524	587	70
-9	65	-14	66	-2	-17	-12	-28	0	4	-6	7	2	-6	5	3	279	736	145	353	448	309	-285	465
8	-2	114	-5	13	-2	22	10	-2	-4	0	-8	1	0	2	1	-173	-623	47	-1008	-214	-38	135	174
-21	65	-4	68	-8	-21	-4	3	0	2	5	-5	4	6	-8	4	-27	628	69	12	478	323	-161	644
-13	15	-9	79	65	-4	6	-17	3	-3	4	-1	-6	-4	-7	-2	320	264	787	-203	-56	385	-1043	221
-5	85	11	-12	-17	-8	-22	32	-6	-3	-3	5	5	-8	3	-8	-350	190	-886	337	396	-739	739	346
-3	6	-12	31	58	7	11	-14	2	0	5	-1	-8	6	-1	3	275	44	487	44	-262	33	-657	-51
-12	-19	-14	25	85	-12	0	-2	-8	-6	-6	-7	0	-6	2	3	311	-93	699	-289	-462	116	-922	-51

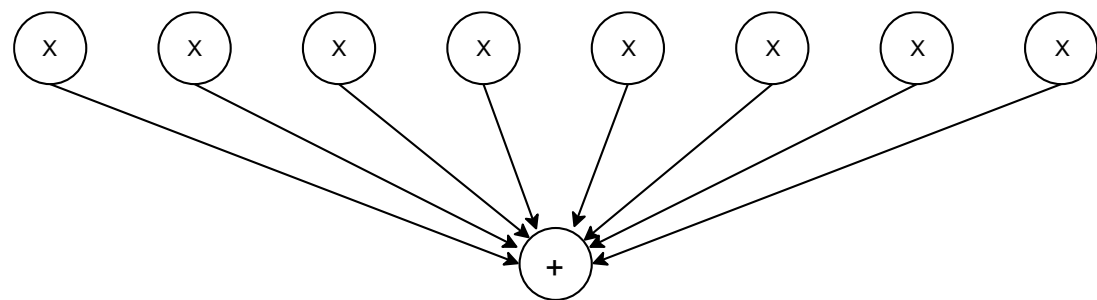
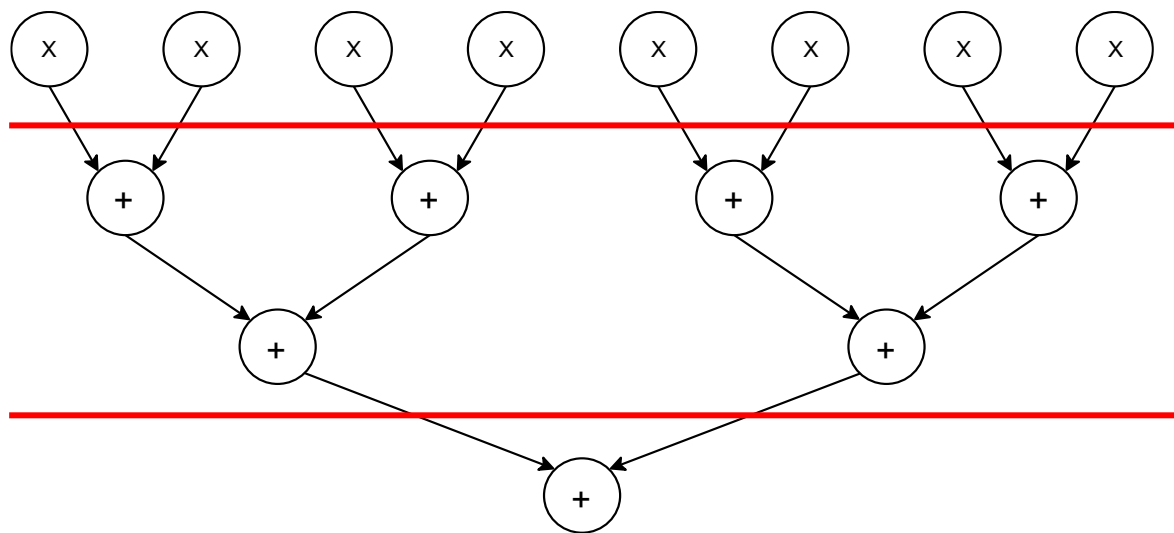
Optimization

- 此次 HW 中, latency, clock period, area 的方面都可以優化

Performance: 30%

Ranking formula: $\text{execution cycles} \times \text{clock period} \times (\text{area})^2$ (less is better)

- latency: get_value 可以自行拉 high, 更早拿到 v, 可以更早算出 output
- clock period: pipeline $\rightarrow$ 一個 cycle 最多做多少事, e.g. (x) & (8個+), (x) 和 (+) 分開



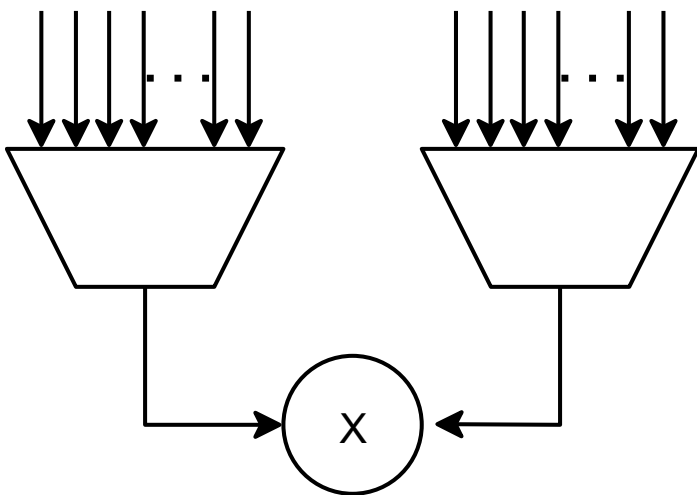
Optimization

- 此次 HW 中, latency, clock period, area 的方面都可以優化

Performance: 30%

Ranking formula: $\text{execution cycles} \times \text{clock period} \times (\text{area})^2$ (less is better)

– area: 共用乘法器, register 要存幾個數



```
1  assign product_temp = in_a * in_b;
2
3  always @(*) begin
4      if (state_0) begin
5          in_a = r1;
6          in_b = r2;
7      end
8      else if (state_1) begin
9          in_a = r3;
10         in_b = r4;
11     end
12     else if (state_2) begin
13         in_a = r5;
14         in_b = r6;
15     end
16     else begin
17         in_a = r7;
18         in_b = r8;
19     end
20 end
```

for loop 使用

- 大量 logic 需要 reset

```

41 logic [15:0] reg [7:0][7:0];
42 logic [15:0] reg_temp [7:0][7:0];
43
44 always @(posedge clk or negedge rst_n) begin
45     if (!rst_n) begin
46         for (int i=0; i<8; i++) begin
47             for (int j=0; j<8; j++) begin
48                 reg[i][j] <= 0;
49             end
50         end
51     end
52     else begin
53         reg <= reg_temp;
54     end
55 end

```

- 重複運算邏輯

```

24 always @(*) begin
25     for (int i=0; i<8; i++) begin
26         product_temp[i] = in_a[i] * in_b[i];
27     end
28 end
29
30 always @(*) begin
31     product_temp[0] = in_a[0] * in_b[0];
32     product_temp[1] = in_a[1] * in_b[1];
33     product_temp[2] = in_a[2] * in_b[2];
34     product_temp[3] = in_a[3] * in_b[3];
35     product_temp[4] = in_a[4] * in_b[4];
36     product_temp[5] = in_a[5] * in_b[5];
37     product_temp[6] = in_a[6] * in_b[6];
38     product_temp[7] = in_a[7] * in_b[7];
39 end

```